小さなプログラムから一つの作品へ



ライフゲームで学ぶ

はじめての Bend2

Bend 2 の文法・並列計算・証明を、ライフゲームで学ぶ

本文・演習・Bend ソース・独立テスト付き

第 1 版　/　Bend 2 基本文法

本書の読み方と検証状態

本書で作るものは、**8×8 のトーラス型ライフゲーム（B3/S23）を 5 世代表示するターミナル・シミュレータ**です。完成版はsrc/08_life_terminal.bendに収録しています。ただし、初めから完成版を丸写しするのではありません。まず Bend 2 の型・関数・配列などを学びます。その後ライフゲームの規則を確認し、1 セルの更新、盤面と近傍の計算、世代更新、文字表示を順に組み合わせます。

最初に Bend 2 の値・型・関数などを学び、次にライフゲームを作り、CPU/GPU 並列実行と形式的証明を学びます。各章では「出力を予想する → 実行する → 改造する → 結果を確認する」という順序で進めてみましょう。

要点

付属テストの結果：Bend 2.0.37 によるサンプルの型検査に成功しています。ライフゲームの生死規則 18 ケースと 5 世代の盤面も、別途作成した Python の参照実装と照合済みです。**テストは、調べた入力について正しい結果が得られたことを示すもの**です。

各章の演習では、まず自分で考えてから付属のサンプルを実行してみましょう。分からない用語は直前の解説や用語集を参照できます。

同梱ファイル

ファイル	内容
src/*.bend	型・関数・配列などの入門用例と、ライフゲーム本体、並列呼び出しと証明の例。
`src/12_affine_probe_INVALID.bend`	意図的に型エラーを起こす練習用コード。
scripts/reference.py	期待する盤面を独立に生成する Python プログラム。
`scripts/test_bend_runtime.py`	Bend コードを実行し、出力の不一致や実行失敗を検出するテスト。
tests/	ライフゲームの参照テストと教材の構成検査。
Bend2_Life_Textbook.tex	本文の LaTeX 原稿。

学習前に知っておきたい基本用語

プログラミングを始めたばかりの方も、本書を読み進められるようにします。ここでは、まず**プログラム**が「コンピュータに行わせる処理を書いたもの」であることを確認します。そのプログラムを人が読んだり編集したりできる文字列が**ソースコード（source code）**です。難しい言葉をすべて覚える必要はありません。コードに出てきたときに、このページへ戻ってください。

用語	意味と本書での使い方
値（value）	数値の `3` や文字列の`"Life"` など、計算の対象になるものです。
変数（variable）	値に付ける名前です。`x` や `board` などを使います。Bend 2 では利用回数に制約がある場合があります。
式（expression）	計算して値になる記述です。たとえば `(2 + 3 : U32)` です。
関数（function）	入力を受け取り結果を返す処理です。`calculate` は整数の計算を行います。
引数（argument）	関数に渡す具体的な値です。`calculate(2,3)` の `2` と `3` が引数です。関数を宣言するときの `x` などの名前は**仮引数（parameter）**と呼びます。
返り値（return value）	関数が計算の結果として返す値です。`calculate(2,3)` の返り値は `13` です。
型（type）	値の種類と、可能な操作を表す情報です。`U32` は 32 ビット符号なし整数、`String` は文字列の型です。
配列（array）	複数の値を順番に並べ、位置を指定してアクセスする構造です。64 要素に固定された仕組みではありません。
添字（index）	配列の要素がどの位置にあるかを示す番号です。最初の位置を 0 として数えます。
再帰（recursion）	関数が自分自身を呼ぶ処理です。終わる条件を設ける必要があります。
副作用（side effect）	値を返すだけでなく、画面表示やファイル書き込みなど外部へ働きかけることです。
コンパイル（compilation）	ソースコードを、実行できる形式などへ変換する工程です。**型検査（type checking）**は、各式の型が合っているかを調べる検査です。
並列計算（parallel computation）	独立した計算を複数同時に進めることです。
形式的証明（formal proof）	法則を対象のすべての値について示し、その根拠を処理系で検査できる形にしたものです。
テスト（test）	期待した入力と出力を用意し、実際の動作が一致するかを調べることです。すべての入力に対する数学的な証明とは異なります。

要点

Bend の型名や構文（`U32`、`Data`、`IO` など）は、英語の綴りのままコードに書きます。

Contents

本書の読み方と検証状態 ii

学習前に知っておきたい基本用語 iii

I Bend 2 の基本文法を学ぶ 1

1 環境構築と最初の実行 2
- 1.1 導入前に確認すること . . . 2
- 1.2 Bend のプログラムを実行する . . . 2
- 1.3 型エラーの表示を確認する . . . 3

2 値とデータ型を理解する 4
- 2.1 値と型の対応を読む . . . 4
- 2.2 選択肢を新しい型として定義する . . . 4

3 関数を定義して数値を計算する 6
- 3.1 引数と返り値の型 . . . 6
- 3.2 計算式の型注釈を確認する . . . 6

4 パターンマッチで場合分けをする 8
- 4.1 データ型の形を見て処理を選ぶ . . . 8
- 4.2 条件に応じて処理を分ける方法 . . . 8
- 4.3 真の場合と偽の場合を追う . . . 9

5 配列の操作とアフィン性 10
- 5.1 配列から値を取り出す基本操作 . . . 10
- 5.2 値の所有権と利用回数 . . . 11

6 List と Maybe：再帰的データと失敗を扱う 12
- 6.1 リストの構造と配列との違い . . . 12
- 6.2 Maybe と Result の違い . . . 12

7 高階関数とクロージャを使う 14
7.1 関数も引数にできる . . . 14
7.2 クロージャは外側の値を覚えている . . . 14
7.3 一度使う関数と、何度も適用する処理を区別する . . . 15

8 モナドと `do` 構文を理解する 16
8.1 値を返す計算と、失敗し得る計算の違い . . . 16
8.2 手続き型の失敗判定を `Maybe` に置き換える . . . 16
8.3 モナドとは何を共通化した仕組みか . . . 17
8.4 `bind` を直接書いてみる . . . 17
8.5 モナドの規則と `Result` への発展 . . . 17
8.6 同じ `do` で入出力をつなぐ . . . 18

9 テンプレートとコンパイル時の引数 19
9.1 `~` は実行時のクロージャではない . . . 19
9.2 コンパイル時に処理を選択する . . . 19

10 数値・文字列・演算子と標準ライブラリ 20
10.1 主要な値の型 . . . 20
10.2 演算子の型付けとビット演算 . . . 20
10.3 標準ライブラリを調べる . . . 20

11 モジュールと複数ファイルのプログラム 22
11.1 `import ./... as M` . . . 22
11.2 モジュール名を使って関数を呼ぶ . . . 22

II ライフゲームを設計して組み立てる 24

12 ライフゲームのしくみと制作するもの 25
12.1 何を作るか . . . 25
12.2 生死規則 B3/S23 . . . 25
12.3 受け入れ条件 . . . 26

13 座標と index 関数で 8×8 盤面を扱う 27
13.1 生死規則を関数で表す . . . 27
13.2 座標から格納位置を求める関数 . . . 28
13.3 すべてのセルを 0 で初期化する . . . 28
13.4 トーラスでも座標から考える . . . 28

14 グライダーを初期配置する 29

14.1 小さなパターンを正確に配置する 29
14.2 盤面を数える 30

15 トーラスの座標と 8 近傍 31
15.1 盤面の端を越えた座標を計算する 31
15.2 周囲 8 セルを読み取り生存数を数える 31
15.3 境界にいるセルを手計算する 33

16 全セルを同期的に更新する 34
16.1 更新に必ず二枚の盤面を使う 34
16.2 再帰処理が終わる条件を決める 34
16.3 ソースを読む順番 36
16.4 別の実装と結果を比較する 37

17 複数世代の実行とターミナル描画 38
17.1 世代を繰り返す関数 38
17.2 出力を作る関数は更新と分ける 38
17.3 実行する 41
17.4 期待される世代 4 41

18 テスト設計と失敗の切り分け 42
18.1 プログラムの正しさをテストで確認する 42
18.2 参照実装を動かす 42
18.3 用意された参照テスト 6 件 43

19 独立した計算を CPU で並列に実行する 44
19.1 二つの関数呼び出しを同時に始める 44
19.2 ライフゲームを二つ同時に計算する 45

20 GPU で計算を実行する 46
20.1 関数名の後ろの! を読む 46
20.2 GPU の有無を分けて確かめる 47

21 ライフゲームの規則を形式的に証明する 48
21.1 法則と証明を対応させる 48
21.2 帰納法と証明の検査 49
21.3 盤面全体の仕様を証明するには 50

22 完成版の設計をレビューする 51
22.1 関数が担う責務 51

22.2 保守性チェック 51

III 演習の解答と実行リファレンス 52

23 重要演習の解答と検証方法 53

23.1 18 ケースの局所規則 53
23.2 代表的な添字 53
23.3 グライダーの変位 53
23.4 近傍読み出し回数 53
23.5 典型的なバグ 54

24 環境・コマンド・参照先 55

24.1 コマンド早見表 55
24.2 主要資料 55
24.3 検証ログの書式 55
24.4 最後の総合チェック 56

PART I

Bend 2 の基本文法を学ぶ

この Part の目次

1 環境構築と最初の実行 2

2 値とデータ型を理解する 4

3 関数を定義して数値を計算する 6

4 パターンマッチで場合分けをする 8

5 配列の操作とアフィン性 10

6 List と Maybe：再帰的データと失敗を扱う 12

7 高階関数とクロージャを使う 14

8 モナドと do 構文を理解する 16

9 テンプレートとコンパイル時の引数 19

10 数値・文字列・演算子と標準ライブラリ 20

11 モジュールと複数ファイルのプログラム 22

要点

Bend 2 の値と型を理解したら、関数、場合分け、配列の操作、リスト、高階関数、クロージャ、モナド、モジュールへ順に進みます。小さなプログラムを動かしながら、構文と結果を確認してみましょう。

Chapter 1

環境構築と最初の実行

本章では、**ターミナル（terminal）**という文字入力の画面から Bend を起動します。コマンドを一つ実行し、表示が変わるところまで進みましょう。

1.1　導入前に確認すること

本書では Bend 2 を使用します。旧 Bend 1 と構文が異なるため、インストールしたコンパイラの `bend version` を確認しましょう。Windows では WSL の Linux 環境を利用できます。macOS や Linux ではターミナルから操作します。

```
curl -fsSL https://bend-lang.com/install.sh -o install-bend.sh
# Inspect install-bend.sh before execution.
sh install-bend.sh
export PATH="$HOME/.bend/bin:$PATH"
bend version
bend guide
bend base
```

Listing 1.1: 導入、動作確認、一次資料の表示

`bend version` の値と表示されたパスをVERSION_LOG.mdへ記録します。インストール用シェルを内容確認せず実行しません。`bend update` がある環境では勝手に更新した後の資料と旧コードを混ぜず、バージョンを記録してから学習します。

1.2　Bend のプログラムを実行する

まず、画面に短い文章を表示するプログラムを動かします。次の例を見てください。分からない記号があっても、今はすべて暗記する必要はありません。

```
import Base

def main() -> IO(Unit):
  do IO<Unit>:
    IO.print("Hello, Bend 2")
```

Listing 1.2: はじめての Bend 2（01_hello.bend）

このコードが行うのは、**画面に「Hello, Bend 2」と表示すること**だけです。上から一行ずつ見ていきましょう。

1. `import Base` は、Bend に用意されている基本的な機能を使えるようにします。ここでは文字を表示するために使います。
2. `def main() -> IO(Unit):` は、プログラムの開始点となる `main` という名前の処理を定義します。`def` は処理を定義するための言葉です。`->` 以降は処理の結果に関する型の指定ですが、詳しい読み方は後の章で学びます。
3. `do IO<Unit>:` は、画面への表示などを順に実行する部分の始まりです。前の行より右に字下げされた行が、この部分に含まれます。
4. `IO.print("Hello, Bend 2")` が、括弧の中の文章を画面に表示します。ダブルクォーテーション `"` で囲んだ部分が表示する文字列です。

IO（入出力）とは、画面への表示やキーボードからの入力など、プログラムの外とのやり取りです。`Unit` は「特に返す値がない」ことを表す型です。どちらも後の章で詳しく学びます。

ターミナルで、このリポジトリのフォルダーを開き、次の一行を入力してください。

```
bend src/01_hello.bend
```

Listing 1.3: 最初のプログラムを実行する

`bend` はプログラムを動かすコマンド、`src/01_hello.bend` は実行するファイルの場所です。成功すると、次のように表示されます。

```
Hello, Bend 2
```

Listing 1.4: 画面に表示される内容

ここまでできたら、src/01_hello.bendの`"Hello, Bend 2"` を好きな文章に変えて保存し、もう一度同じコマンドを実行してみましょう。表示が変われば成功です。

要点

実行せずにプログラムの書き方を確認したいときは、`bend src/01_hello.bend --check-only` を使えます。ネイティブ実行ファイルを作る方法もありますが、最初は上の `bend` コマンドだけで十分です。

1.3 型エラーの表示を確認する

`IO.print` の文字列を数値に変えてみましょう。次に型注釈を消してみます。さらに返り値の型を `U32` にして表示命令を残し、どのエラーが出るか観察します。エラーメッセージのうち **expected** と **observed** がどの式を指しているかを説明します。

ハンズオン／確認問題

(1) `Hello, Bend 2` の文字列を変更して実行してみましょう。(2) `bend guide` で「Types」に関する項目を探してみましょう。(3) 導入版・OS・CPU・Clang の情報をVERSION_LOG.mdに保存してみましょう。**合格条件:** ソースを編集して標準出力を変えられること。

Chapter 2

値とデータ型を理解する

プログラムを書く前に、**値（value）**と**型（type）**を区別しましょう。Bend 2 の型は「どんな値を表し、何をしてよいか」を決めます。

2.1 値と型の対応を読む

データ型（data type）とは、値の種類を定めるものです。例えば `42` は数値、`"hello"` は文字列です。Bend 2 で頻出する型を先に見ておきます。

型	値の例	用途
`Nat`	`3n`	0 以上の自然数を表します。
`U32`	`42`	0 から $2^{32}-1$ までの符号なし整数です。
`Bool`	`True{}`	真または偽を表します。
`String`	`"hello"`	文字列を表します。
`F32`	32 ビット浮動小数点	小数を近似して扱う数値型です。

同じ数字に見えても `3n` と `3` は異なる型の値です。Bend 2 は型の暗黙変換が少ないため、型を意識してコードを読む必要があります。**型注釈（type annotation）**は、値や式の型を明示する記法です。例えば `(3 + 4 : U32)` は、`U32` 型の足し算を表します。

2.2 選択肢を新しい型として定義する

既存の型だけでは表しにくい値は、**ユーザー定義型（user-defined type）**を作って表現できます。次の例では、**コンストラクタ（constructor）**の `Idle{}` と `Busy{}` という二種類だけの状態を定義します。コンストラクタとは、**その型の値を組み立てるための名前**です。

```
import Base

# A custom datatype with two constructors.
type Status is Data:
  Idle{}
  Busy{}

def main() -> Status:
  Idle{}
```

Listing 2.1: 二つの状態を持つデータ型（28_data_types.bend）

最初の `type Status is Data:` が型の宣言、続く二行が作れる値の候補です。最後の `main` はプログラムの入口で、詳しい関数の書き方は次章で学びます。ここでは、`Idle{}` が `Status` 型の値になることを読み取りましょう。

要点

代数的データ型（algebraic data type; ADT）は、候補を列挙したり、候補ごとに別のデータを持たせたりできる型です。例えば `Some{value}` と `None{}` で「値がある／ない」を表す `Maybe` もその一例です。`is Data` は値を再利用できる分類を表しますが、所有権とあわせて後の章で扱います。

ハンズオン／確認問題

`Status` に `Paused{}` を追加してみましょう。`Idle{}` と `Busy{}` 以外の状態を、型を変えずに作れるか考えてみましょう。

Chapter 3

関数を定義して数値を計算する

関数（function）、**引数（argument）**、**返り値（return value）**を短い数値計算で確認します。特定のアプリケーションに依存しない例で、型注釈と式の読み方を身につけましょう。

3.1 引数と返り値の型

Bend では `def f(x: T) -> R:` の形で関数を宣言します。戻り値は最後の式であり、Python のように常に `return` が必要なのではありません。`U32` は 0 以上の整数を 32 ビットに格納する型です。`Nat` は 0、1、2、……という**自然数（natural number）**の型で、再帰や型の添字に使います。`3n` は `Nat` のリテラルで、`3` とは扱いを区別します。

```
import Base

# General arithmetic example independent of any application.
def calculate(+x: U32, y: U32) -> U32:
  (x * 5 + y : U32)

def main() -> U32:
  calculate(2, 3)
```

Listing 3.1: 整数の乗算と加算（02_arithmetic.bend）

例の関数 `calculate(x,y)` は $5x + y$ を計算します。`calculate(2,3)` の結果は 13 です。`(x * 5 + y : U32)` では、計算式の後ろに型を示しています。`+x` の `+` は「値を複数回使える」と示す**使用量（quantity）**の指定であり、この短い例では必須ではありません。詳しくは配列とアフィン性の章で扱います。

3.2 計算式の型注釈を確認する

整数加減算の演算子だけを式として書かず、`(x + y : U32)` のように型を与える。`Nat` と `U32` の暗黙変換を当てにしません。`U32.to_nat`・`U32.from_nat` 等は `bend base` で確認します。`U32` には範囲の限界があり、任意精度整数として扱ってはいけません。

要点

数字だけで動作確認する `main() -> U32` は、値の正規化と表示が行われる経路になります。実行可能ファイルを生成する場合は`-o` オプションを使います。

ハンズオン／確認問題

`calculate(0,0)`、`calculate(1,2)`、`calculate(2,3)` の結果を予想してから実行してみましょう。式の 5 を 10 に変えると何が変わるでしょうか。また、定数 5 を引数 `factor: U32` に変える方法も考えてみましょう。

Chapter 4

パターンマッチで場合分けをする

Bend 2 では、条件に応じて処理を分けるときに**パターンマッチ（pattern matching）**を使います。ここでは、真偽値に応じて二つの数字から一つを選ぶ、一般的な例で基本形を学びます。

4.1 データ型の形を見て処理を選ぶ

前章で、`Status` には `Idle{}` と `Busy{}` という値の候補があることを学びました。**パターンマッチ（pattern matching）**は、**値がどの形に当てはまるか**を調べ、形ごとに処理を分ける仕組みです。`Bool` の `True{}` ／ `False{}` も同じ考え方で処理できます。

```
import Base

# A custom type restricts the set of valid values.
type Status is Data:
  Idle{}
  Busy{}

def to_u32(status: Status) -> U32:
  match status:
    case Idle{}:
      0
    case Busy{}:
      1

def main() -> U32:
  to_u32(Busy{})
```

Listing 4.1: ユーザー定義型をパターンマッチする例（13_status_adt.bend）

`case Idle{}:` なら 0、`case Busy{}:` なら 1 を返します。ここでは数値を返していますが、選択肢の種類が増えたら対応する場合分けも追加する必要があります。

4.2 条件に応じて処理を分ける方法

たとえば「条件が真なら 10、偽なら 20 を返す」という処理には、条件による場合分けが必要です。`match` を使うと、それぞれの条件に対応する処理を明確に書けます。

Bend 2 の公開ガイドには通常の `if ... else` 構文がなく、`match` と `True{}` ／ `False{}` を使います。特に**計算した式を直接 match できない**制限があります。たとえば比較式をそのまま `match` せず、

計算結果を引数として受け取った関数の中で場合分けします。

```
import Base

# General-purpose conditional choice; no Life-specific rules.
def choose(flag: Bool, first: U32, second: U32) -> U32:
  match flag:
    case True{}:
      first
    case False{}:
      second

def main() -> U32:
  choose(True{}, 10, 20)
```

Listing 4.2: 真偽値による場合分け（27_match_basics.bend）

choose に渡す真偽値 flag が True{} なら最初の数値を、False{} なら次の数値を返します。match は**引数として受け取った値**に対して行います。条件が何を意味するかは呼び出す側が決めるため、さまざまな処理で再利用できます。

4.3　真の場合と偽の場合を追う

条件	第 1 候補	第 2 候補	返り値
True{}	10	20	10
False{}	10	20	20
True{}	5	9	5

ハンズオン／確認問題

choose(True{},10,20) と choose(False{},10,20) の返り値を予想して確かめてみましょう。第 1 候補と第 2 候補を変えて結果を比較してみましょう。比較式 U32.is_eq の結果を choose へ渡す使い方も試してみましょう。

Chapter 5

配列の操作とアフィン性

ここでは**配列（array）**を学びます。配列とは、複数の値を順番にまとめ、位置を指定して扱うデータ構造です。あわせて、Bend 2 の**アフィン性（affinity）**という値の使用方法に関する制約を学びます。

5.1　配列から値を取り出す基本操作

配列の要素を読むには、`Array.get` を使います。ここで一つだけ、普通の配列操作と違う点があります。Bend 2 では、通常の配列は所有する値として扱われ、操作に渡すと元の変数をそのまま再利用できません。そこで `Array.get` は、**取り出した要素**だけでなく、**その後も使うための配列**も組で返します。まずこの形を見てから、なぜ所有権が必要なのかを確認しましょう。

`Array<U32>` は、32 ビット符号なし整数を格納する配列型です。**種類（kind）**とは「型を分類するための分類」で、`Type` は値を自由に複製できない種類、`Data` は複製を許せる種類を表します。`Array<U32>` は `Type` に属するため、同じ配列を二つの処理に勝手に渡せません。読み取るときは `Array.get` が**組（pair）**として「続けて使う配列」と「読み取った値」の両方を返します。こうすることで同じ配列を勝手に何度も使わず、読み取り後の配列を再び明示して処理できます。

```
import Base

def read_value(r: Array<U32> & U32) -> U32:
  (a, value) = r
  value

def main() -> U32:
  a = [0 : U32*8n]
  a = Array.set(U32, a, 5, 42)
  read_value(Array.get(U32, a, 5))
```

Listing 5.1: 書き換えと読み出し（04_array.bend）

`[0 : U32*8n]` は、0 を 8 個並べて作る配列です。**8 要素という長さはこのサンプルが選んだ値**であり、Bend 2 の配列が 8 要素に限定されるわけではありません。`Array.set(U32,a,5,42)` は、添字 5 に 42 を書き込みます。続く `Array.get` は、`(a,value)` という組を返します。ただし計算結果をその場で直接分解せず、`read_value` 関数の引数 `r` として受け取り、関数の中で分解しています。最後は `value` だけを結果とするので、`a` は破棄されます。Bend は**アフィン**（高々 1 回の使用）であり、破棄は許されます。後出の「配列操作の糖衣構文」では `a[i]` と `a[i] <- v` も紹介します。

注意

Array は +a を宣言して何度も使うことができません。src/12_affine_probe_INVALID.bendはこの失敗を観察する専用ファイルであり**正しくないことが正常**。普通の実行対象には含めない。

5.2 値の所有権と利用回数

記法	解釈
x: T	変数 x を高々 1 度使う。使わなくてもよい。
+x: T	T が Data なら何度も使えます。参照カウント等のコストは無視できません。
-A: Data	消去される型引数。計算結果の型検査には現れるが実行時には不要。
Array<T>	Type に属し、通常はコピー可能な Data ではありません。

ハンズオン／確認問題

Array.get の (array,value) を (value,array) と逆にするとどうなるか。Array.set した配列を使わず、古い配列を再利用すると何が問題か。エラーの全文を記録し、型エラーと規則違反を区別してみましょう。

Chapter 6

List と Maybe：再帰的データと失敗を扱う

配列以外のデータの表し方を学びます。**リスト（list）**は要素を先頭からつなぐ構造で、Maybe は「結果があるかどうか」を型に含める仕組みです。

6.1　リストの構造と配列との違い

Array は長さが 2 の冪などの制約を持つ所有権付き領域であり、List<A> は Nil{} または Con{h,t} で構成される帰納的な鎖です。処理する数値の並びや、順番に発生するイベントはリストで表現できます。

```
import Base

# A List is an inductive structure. The tail is smaller.
def count(xs: List<U32>) -> Nat:
  match xs:
    case Nil{}:
      0n
    case Con{h, t}:
      (1n + count(t) : Nat)

def main() -> Nat:
  count([1, 10, 16, 17, 18])
```

Listing 6.1: リストを構造再帰で処理する（14_list_basics.bend）

6.2　Maybe と Result の違い

Maybe<A> は値が Some{value} か None{} かを表します。Result は成功または失敗の情報を保持する用途に適します。文字列を数値に変換する場面では、0 を「解析失敗」の印と兼用せず、失敗を型で表す方が安全になります。Maybe の正確な量パラメータは bend base Maybe で確認します。

要点

再帰関数は**停止性を証明できる構造的な減少**が重要です。List では Con の tail、Nat では 1n+p の p が自然な減少引数になります。

ハンズオン／確認問題

(1) 整数を数個並べた `List<U32>` の長さを数えてみましょう。(2) 空リストでは長さが 0 になることを確かめてみましょう。(3) 数値の解析に失敗した場合を `Maybe` で表現する方法を考えてみましょう。

Chapter 7

高階関数とクロージャを使う

手続き型言語では、関数を「処理を呼び出す手段」と考えることが多いでしょう。ここでは、**関数そのものを値として受け渡す**考え方を学びます。その際に必要になる**高階関数（higher-order function）**と**クロージャ（closure）**を区別します。

7.1　関数も引数にできる

高階関数とは、**関数を引数として受け取るか、結果として関数を返す関数**です。例えば `apply` は「どの処理をするか」を引数 `f` として受け取り、入力 `x` に適用します。`U32 -> U32` は、`U32` を受け取り `U32` を返す**関数型（function type）**です。

```
import Base

# Passing a function as an input.
def apply(f: U32 -> U32, x: U32) -> U32:
  f(x)

def main() -> U32:
  apply(x => (x + 2 : U32), 40)
```

Listing 7.1: 関数を引数に渡す（29_higher_order.bend）

`x => (x + 2 : U32)` は**無名関数（anonymous function）**です。名前を付けずにその場で関数を作ります。この例では 40 を受け取り 42 を返します。手続き型言語の「処理の分岐を増やす」設計とは異なり、**共通の処理手順を残したまま、実際の計算だけを差し替える**ことができます。

7.2　クロージャは外側の値を覚えている

無名関数が外側の変数を使う場合、関数本体だけでは結果を決められません。そこで、**クロージャは関数と、その関数が利用する外側の値（捕捉した値）をまとめたもの**として動きます。

```
import Base

# A captured function is affine, even if the captured value is Data.
def adder(k: U32) -> U32 -> U32:
  x => (x + k : U32)

def main() -> U32:
  add2 = adder(2)
```

```
add5 = adder(5)
add5(add2(1))
```

Listing 7.2: 異なる値を捕捉する二つのクロージャ（15_closure.bend）

`adder(2)` の結果である `add2` は、`k=2` を覚えた関数です。`add2(1)` は 3 になり、`add5(3)` は 8 になるので、全体では 8 を返します。`adder` は**関数を返す高階関数**にもなっています。`U32.add(2)` のように一部の引数を先に与える**部分適用（partial application）**もクロージャの一例です。

注意

Bend 2 では、**クロージャはアフィン**です。つまり、同じクロージャを呼べるのは高々 1 回です。捕捉した `U32` が複製可能でも、クロージャ自体を何度も呼ぶことはできません。一方、`adder` のようなトップレベルの関数定義は繰り返し呼び出せます。これは Python や JavaScript の一般的なクロージャとは重要な違いです。

7.3　一度使う関数と、何度も適用する処理を区別する

`apply(f, x)` は `f` を一度使うので、通常の高階関数として書けます。しかし、同じ `f` を二度使う `f(f(x))` は、一般のアフィンな関数引数では書けません。次章では、**コンパイル時にコードを埋め込むテンプレート**を学び、この違いを確かめます。

要点

高階関数は「関数を受け取る／返す」という**役割**の名前です。**クロージャ**は「関数と捕捉した値」を一体として扱う**値**の名前です。同じものを指す言葉ではありません。

ハンズオン／確認問題

`29_higher_order.bend` で無名関数を `x => (x * 2 : U32)` に変えて結果を予想してみましょう。続いて `adder(3)` を作り、捕捉する値によって結果がどう変わるか確認してみましょう。同じクロージャを二度呼ぶ例は別ファイルにして、型検査が拒否する理由も考えてみましょう。

Chapter 8

モナドと do 構文を理解する

この章では、手続き型言語の「一行ずつ処理する」書き方と、関数型言語で**計算の文脈をつなぐ**書き方の違いを学びます。特に、失敗するかもしれない数値変換を例に**モナド（monad）**を理解します。

8.1　値を返す計算と、失敗し得る計算の違い

`(2 + 3 : U32)` は必ず `U32` を返す計算です。一方、文字列を整数に変える `U32.read("42")` は成功するとは限りません。`"abc"` を渡した場合、返すべき整数がありません。**`Maybe`** はこの違いを型で示します。成功なら `Some{42}`、失敗なら `None{}` です。`Maybe<&2, U32>` という型は、**`U32` が得られるかもしれない計算結果**を表します。先頭の `&2` は値の再利用可能性に関する Bend 2 固有の量指定です。ここでは「値があるかないか」と別の役割であることを押さえてください。

8.2　手続き型の失敗判定を `Maybe` に置き換える

従来の言語では、整数を読み取った後に成否を調べ、失敗ならそこで処理を止める `if` 文を何度も書くことがあります。`Maybe` なら**失敗の伝播（failure propagation）**を共通の仕組みに任せられます。

```
import Base

# A failed read (None) short-circuits the Maybe block.
def add_strs(a: String, b: String) -> Maybe<&2, U32>:
  do Maybe<&2, U32>:
    x : U32 <- U32.read(a)
    y : U32 <- U32.read(b)
    return (x + y : U32)

def main() -> Maybe<&2, U32>:
  add_strs("40", "2")
```

Listing 8.1: 二つの文字列を安全に加算する（21_monadic_maybe.bend）

実行例は `add_strs("40", "2")` が `Some{42}`、`add_strs("abc", "2")` が `None{}` です。**最初の解析に失敗した場合、次の解析と加算へ進みません。**失敗を表すために 0 や負数などの「特別な数値」を予約する必要もありません。

8.3 モナドとは何を共通化した仕組みか

モナドは、値を包む型の仕組みと、**その型を保ったまま複数の計算をつなぐ規則**を組にした抽象化です。Bend 2 では、例えば Maybe に次の二つの操作があります。

操作	役割
M.pure	普通の値を M の計算結果として包みます。Maybe なら Some{x} に相当します。
M.bind	包まれた値を次の関数へ渡し、次の計算も M の形で返します。Maybe なら None{} を次へ進めず、そのまま返します。

ここで M は Maybe や IO などの型を表す仮の名前です。「モナド＝入出力」でも「モナド＝副作用」でもありません。Maybe は失敗、Result は失敗の理由、IO は外部とのやり取りという**異なる文脈**を扱います。

8.4 bind を直接書いてみる

do は新しい種類の計算を実行する命令ではなく、bind と pure を読みやすく書くための**構文糖（syntactic sugar）**です。次のコードでは、解析に失敗したときの処理を Maybe.bind へ任せています。

```
import Base

# Expanded form of the do-notation example.
def add_strs_explicit(a: String, b: String) -> Maybe<&2, U32>:
  Maybe.bind(&2, U32, U32, U32.read(a), x =>
    Maybe.bind(&2, U32, U32, U32.read(b), y =>
      Maybe.pure(&2, U32, (x + y : U32))))

def main() -> Maybe<&2, U32>:
  add_strs_explicit("40", "2")
```

Listing 8.2: Maybe.bind を直接使う（30_monad_explicit.bend）

このプログラムと、前節の do Maybe<&2, U32>: を使ったプログラムは同じ結果を返す設計です。x : U32 <- U32.read(a) は「成功した値を x へ受け渡して続行する」という意味であり、**通常の変数代入（x = ...）とは違います。**

要点

do M<..., R>: の x : A <- m は、M.bind(..., A, R, m, x => ...) を使う形へ展開されます。return e は M.pure(..., R, e) を使う形へ展開されます。ここでの return は、C 言語の return のような「命令的な早期脱出」と混同しないでください。

8.5 モナドの規則と Result への発展

一般にモナドとして使う pure と bind は、計算を一貫してつなぐために**左単位律・右単位律・結合律**と呼ばれる規則を満たすように設計します。初めは数式を暗記する必要はありません。「普通の値を包んでから続ける操作」と「そのまま続ける操作」が同じ結果になること、つなぐ場所の括り方を変えても意味が変わらないことが要点です。なお、**pure と bind という名前だけを持つ型が、必ずこの規則を満たすわけではありません。**規則が成り立つかは実装側の責任です。Result

は Done{value} と Fail{error} を使い、失敗の理由を残せます。Maybe は単に失敗したことだけを知らせればよいとき、Result は**何が間違っていたかを説明したいとき**に向いています。

8.6 同じ do で入出力をつなぐ

端末への表示では do IO<Unit>: を使います。IO.print は「文字列を表示するという**効果（effect）**」を記述します。do 内に順番に並べることで、表示の順序を明示できます。**Maybe では失敗を、IO では効果の順序を扱いますが、つなぎ方の構文は共通**です。二つの例で違いを確認してみましょう。

ハンズオン／確認問題

add_strs("12", "30") と add_strs("abc", "30") の結果を予想し、実行してみましょう。do 版と bind 版の結果を比較してみましょう。さらに、三つの文字列を足す関数を作り、途中で失敗した場合の返り値を確かめてみましょう。

Chapter 9

テンプレートとコンパイル時の引数

「実行するたびに値を受け取る」のと「コンパイル時に処理を埋め込む」のは違います。ここでは**テンプレート（template）**を後者として理解します。

9.1 ~ は実行時のクロージャではない

テンプレート・パラメータ ~f はコンパイル時にコードとして代入されます。関数適用が複数必要なときでも、アフィンなクロージャを無理にコピーしません。**各テンプレート引数の組み合わせごとにインスタンス化**されるため、コードサイズとのトレードオフがあります。

```
import Base

# ~f is a compile-time function argument, unlike a closure.
def twice(~f: U32 -> U32, x: U32) -> U32:
  f(f(x))

def main() -> U32:
  twice(~(x => (x + 1 : U32)), 40)
```

Listing 9.1: 同じ処理を 2 度行うテンプレート（16_template.bend）

~(x => (x + 1 : U32)) は**呼び出し元の局所変数を捕捉しない閉じた構文**でなければなりません。テンプレートの宣言順序にも制約があります。List.map など Base の定義を bend base で読むと利用例が分かります。

9.2 コンパイル時に処理を選択する

処理 f を通常の引数にするか、~f としてテンプレート引数にするかは、実行時に選ぶ必要があるか、コンパイル時に決まってよいかで判断します。型と使い回しの制約も比較してみましょう。

ハンズオン／確認問題

twice を thrice へ拡張してみましょう。テンプレートと普通のクロージャの寿命・生成コードの違いを、実行時とコンパイル時に分けて説明してみましょう。

Chapter 10

数値・文字列・演算子と標準ライブラリ

数値型と文字列型は、用途ごとに使い分けます。まず整数と小数の違い、演算子が受け取る型、標準ライブラリの調べ方を整理します。

10.1　主要な値の型

Bend 2 の基本的な数値型は `Nat`、`U32`、`F32` です。`F32` は小数を近似的に表す**浮動小数点数（floating-point number）**で、ビット単位の演算や丸めに注意します。`Bool`、`Cmp`、`Char`、`String` も使用します。現行の公式ガイドは **F64 や I64 が最初から使える**とは述べていない。

```
import Base

# Parenthesized operators carry a type annotation.
def next_index(+i: U32) -> U32:
  ((i + 1 : U32) % 64 : U32)

def main() -> String:
  "next index = " ++ U32.show(next_index(63))
```

Listing 10.1: 整数演算と文字列化（17_numbers_strings.bend）

10.2　演算子の型付けとビット演算

`(a + b : U32)` のように、型注釈は**その演算を囲う式自身**に置きます。`&&`・`||` は Bool、`++` は String。`.&.`、`.|.`、`.^.` や `<<` などは `U32` のビット演算で、一般の数学的な等式と区別します。

10.3　標準ライブラリを調べる

`import Base` で読み込む**標準ライブラリ（standard library）**には、文字列化や配列操作など、よく使う関数が用意されています。`bend base U32` で整数の操作を、`bend base String` で文字列の操作を調べられます。例えば、整数を文字列で表示したいときは `U32.show` を使います。実行して確認してみましょう。

```
bend base U32
bend base String
bend src/17_numbers_strings.bend
```

Listing 10.2: 標準ライブラリの確認例

ハンズオン／確認問題

(1) `Nat` と `U32` が自動変換されない例を探してみましょう。(2) `U32` の AND・OR・XOR と比較の違いを小さい入力で表にまとめてみましょう。(3) 数値を文字列へ変換するときに使用できる関数を調べてみましょう。

Chapter 11

モジュールと複数ファイルのプログラム

一つのファイルが長くなったら、機能ごとに分割します。**モジュール（module）**とは、関数や型をひとまとめにした再利用単位です。

11.1 `import ./... as M`

ソースを複数のファイルに分けると、数値計算、入出力、テストなどの責務を整理しやすくなります。各.bend ファイルがモジュールです。`import ./file.bend as M` は、取り込んだ定義を `M` という名前で区別できるようにします。以下の例では `module_math.bend` を取り込みます。

```
import Base

# A helper module, imported by 19_module_main.bend.
def square(+x: U32) -> U32:
  (x * x : U32)

def cell_index(y: U32, x: U32) -> U32:
  (y * 8 + x : U32)
```

Listing 11.1: 取り込まれる側の定義（module_math.bend）

```
import Base
import ./module_math.bend as Math

def main() -> U32:
  Math.square(Math.cell_index(1, 2))
```

Listing 11.2: 別ファイルから定義を呼ぶ（19_module_main.bend）

```
bend src/19_module_main.bend
# 例: math.bend, parser.bend, main.bend
```

Listing 11.3: モジュールの実行と将来の分割

11.2 モジュール名を使って関数を呼ぶ

同じ名前の関数が別のモジュールにある場合でも、`M.square` のようにモジュール名を付けて呼び出せます。

ハンズオン／確認問題

`square` だけを別の`.bend` へ切り出し、`import` を通して呼べることを確認してみましょう。また、ファイル同士が相互に取り込まない構成を考えてみましょう。

PART II

ライフゲームを設計して組み立てる

この Part の目次

12 ライフゲームのしくみと制作するもの 25
13 座標と index 関数で 8×8 盤面を扱う 27
14 グライダーを初期配置する 29
15 トーラスの座標と 8 近傍 31
16 全セルを同期的に更新する 34
17 複数世代の実行とターミナル描画 38
18 テスト設計と失敗の切り分け 42
19 独立した計算を CPU で並列に実行する 44
20 GPU で計算を実行する 46
21 ライフゲームの規則を形式的に証明する 48
22 完成版の設計をレビューする 51

要点

最初にライフゲームの規則を確認し、盤面の配列、初期配置、近傍の数え方、世代更新、表示、テストを実装します。Part I で学んだ小さな処理を組み合わせて完成させましょう。

Chapter 12

ライフゲームのしくみと制作するもの

本章では、ライフゲームの規則を日本語と数式で整理します。**仕様（specification）**とは、プログラムが満たすべき動作の条件です。先に条件を決めておくと、テストで確認できます。

12.1　何を作るか

ライフゲームは二次元格子の各セルが生存（1）か死亡（0）かを持つ**セル・オートマトン（cellular automaton）**です。周囲 8 方向にある**近傍セル（neighboring cell）**の生存数を数えて、次の状態を決めます。簡単な規則ですが、移動する模様や形を変え続ける模様が生まれます。そのため、小さなプログラムでも**モデル化（modeling）**、境界での処理、テスト、性能評価を学べます。

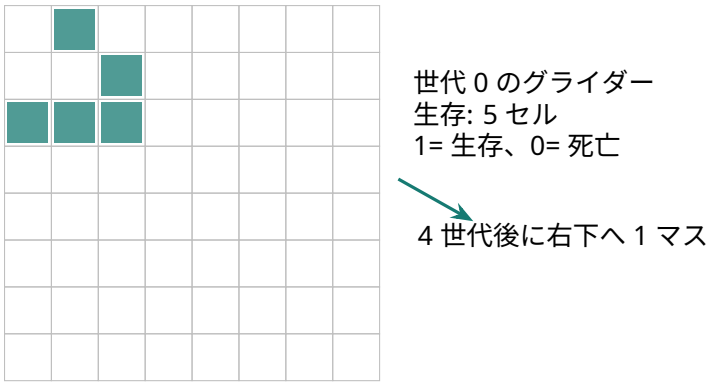
本書では格子を 8 行 ×8 列、セルを `U32` の 0 と 1、座標を (x, y)（左上が $(0, 0)$）とします。盤面の右端は左端に、下端は上端につながります。これを**周期境界条件（periodic boundary condition）**と呼びます。全セルを**同時に更新**します。

12.2　生死規則 B3/S23

セル $a \in \{0, 1\}$ と、その 8 近傍の生存数 $n \in \{0, \dots, 8\}$ に対して

$$F(a, n) = \begin{cases} 1 & n = 3 \quad \text{（生死によらず誕生／生存）} \\ a & n = 2 \quad \text{（現状維持）} \\ 0 & \text{その他} \end{cases}$$

これが **B3/S23**（Birth at 3, Survive at 2 or 3）です。いわゆる「生存セルは 2〜3 近傍で生き残る」という説明と矛盾しません。生存数だけでは次状態は決まらず、$n = 2$ なら現在の状態を参照することに注意してください。



12.3　受け入れ条件

1. 正しいグライダーを初期状態として 5 枚（世代 0〜4）の 8×8 盤面を表示します。
2. 近傍として数えるのは周囲の 8 マス**だけ**です。中心のセル自身は数えません。
3. すべてのセルの新状態を、更新前の盤面だけから計算します。**その場更新（in-place update）**を 1 セルずつ行う方法は採用しません。
4. ブロックが静止、ブリンカーが周期 2、グライダーが 4 世代で斜めへ 1 マス動くことを Python の独立実装と照合します。
5. 表示と更新を分離して、後で表示形式・実行先を交換できる形にします。

ハンズオン／確認問題

上の規則から、現在 0/1 の各場合について近傍数 0〜8 の真理値表（2×9 個）を作ってみましょう。なぜ「3 なら 1、2 なら現状、それ以外 0」の 3 分岐で全部を処理できるのかを説明してみましょう。

Chapter 13

座標と index 関数で 8×8 盤面を扱う

前章で決めたライフゲームの規則を関数にし、続いて盤面を (x,y) で操作する補助関数を作ります。配列の格納方法は補助関数に任せ、ライフゲームの処理へ集中します。

13.1　生死規則を関数で表す

```
import Base

# The computed comparisons are matched by this helper's parameters.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

# 3 neighbors => live; 2 => retain; otherwise dead.
def rule(alive: U32, +neighbors: U32) -> U32:
  choose(U32.is_eq(neighbors, 3), U32.is_eq(neighbors, 2), alive)

def main() -> U32:
  # dead cell with exactly three neighbors is born
  rule(0, 3)
```

Listing 13.1: ライフゲームの生死規則（03_rule.bend）

rule(alive,neighbors) では、neighbors が 3 なら 1、2 なら alive、それ以外なら 0 を返します。choose の内部では、Part I で学んだ match を使っています。+neighbors は同じ数値を複数回比較するための使用量指定です。

ハンズオン／確認問題

rule(0,3)、rule(1,2)、rule(1,4) の返り値を確認してみましょう。現在の状態 0/1 と近傍数 0〜8 の全 18 通りを表にして、期待値と一致するかを調べてみましょう。

13.2　座標から格納位置を求める関数

盤面は 8 行 ×8 列です。生きているマスを (x, y) で指定できれば、「右へ 1」「下へ 1」もそのまま座標で考えられます。内部の保存先を毎回手計算する必要はありません。

```
1 def index(x: U32, y: U32) -> U32:
2   (y * 8 + x : U32)
```

Listing 13.2: 8×8 盤面で共通に使う補助関数

x は左から数えた列、y は上から数えた行です。どちらも 0〜7 の範囲で指定します。例えば index(1, 2) は 17、index(7, 7) は 63 になります。今後は Array.get や Array.set へ index(x, y) を渡しましょう。

要点

盤面の保存形式は長さ 64 の一次元 Array<U32> のままです。index だけが二次元の座標を格納位置へ変換します。この補助関数は**範囲外の座標を検査しません**。0〜7 の有効な x、y を渡す前提で使います。
Bend 2 の Array.get には範囲外の添字が循環する動作がありますが、それを盤面の境界条件の代わりに使ってはいけません。二次元のトーラスは、まず x と y を各々循環させて実現します。

配列を要素に持つ Array<Array<U32>> も型として表せますが、本書の実習では一枚の一次元配列と index で十分です。セルの配置・近傍・世代更新を理解することを優先します。

13.3　すべてのセルを 0 で初期化する

[0 : U32*64n] で 64 個の要素を用意します。書き込みは Array.set で行い、結果として返る配列を次の操作へ渡します。例えば Array.set(U32,b,index(1,2),1) は座標 (1,2) を生存状態にします。値を移す様子は b0,b1,b2,... という変数名にも表れています。

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

添字：上段が 0〜7、次が 8〜15…

13.4　トーラスでも座標から考える

index は有効な座標を格納位置にするだけです。トーラスの右端 (7,0) から一つ右の座標は (0,0) です。座標ごとに循環させた後で index を呼び出せば、配列の添字の循環と盤面のトーラスを混同せずに済みます。

ハンズオン／確認問題

index(7,0) と index(0,1) の値を予想してみましょう。続いて (7,0) の右隣を index で求めるには、先にどの座標へ直すべきか図にして考えてみましょう。

Chapter 14

グライダーを初期配置する

次に、最初の盤面を用意します。小さな生存セルの模様を指定し、正しい位置に置けたかを自分で確認します。

14.1　小さなパターンを正確に配置する

グライダーは生存セル 5 個で右下へ移動する周期 4 の模様。本書の初期配置は次の 5 点です。

$$\{(1,0),(2,1),(0,2),(1,2),(2,2)\}$$

コードではこの五つを `index(x,y)` にそのまま渡します。数値の添字を暗記する必要はありません。

```
import Base

# In an 8x8 board, x is the column and y is the row (both 0..7).
# This helper assumes the caller has already checked the range.
def index(x: U32, y: U32) -> U32:
  (y * 8 + x : U32)

# 8x8, row major; alive at (1,0),(2,1),(0,2),(1,2),(2,2)
def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0, index(1, 0), 1)
  b2 = Array.set(U32, b1, index(2, 1), 1)
  b3 = Array.set(U32, b2, index(0, 2), 1)
  b4 = Array.set(U32, b3, index(1, 2), 1)
  Array.set(U32, b4, index(2, 2), 1)

def read_value(r: Array<U32> & U32) -> U32:
  (b, v) = r
  v

def main() -> U32:
  read_value(Array.get(U32, glider(), index(2, 2)))
```

Listing 14.1: 初期グライダー（05_glider.bend）

図で見れば次の配置です。`1` を生存、`0` を死亡とします。

```
01000000
00100000
11100000
00000000
```

```
00000000
00000000
00000000
00000000
```

Listing 14.2: 世代 0 の配置（8 列 ×8 行）

14.2 盤面を数える

グライダーの生存数は 5。今はセル値を `U32` として持つため、`2` や `42` も表現自体はできます。**セルが 0/1 であるという不変条件**は今のところシードと更新関数がそれ以外を作らないという開発規約に依存しています。後半では「`U32` で表現可能」と「型で 0/1 に限定される」を分けて考えます。

ハンズオン／確認問題

ブロック（$(3,3),(4,3),(3,4),(4,4)$）を初期化する `block()` を作ってみましょう。ブリンカーを $y=3, x=3,4,5$ に置く `blinker()` を作ってみましょう。出力で指定した場所だけが `1` となることを確認してみましょう。

Chapter 15

トーラスの座標と 8 近傍

どのセルが隣かを決めます。今回は画面の右端と左端、上端と下端がつながる**トーラス（torus）**状の盤面です。

15.1　盤面の端を越えた座標を計算する

左隣は $(x-1+8) \bmod 8$、右隣は $(x+1) \bmod 8$。$x \in \{0,\dots,7\}$ に限れば $(x+7) \bmod 8$ で代用できます。同様に上下も $(y+7) \bmod 8$、$(y+1) \bmod 8$ で求まる。負の数を `U32` に入れない。

$$L(x) = (x+7) \bmod 8, \quad R(x) = (x+1) \bmod 8, T(y) = (y+7) \bmod 8, \quad B(y) = (y+1) \bmod 8.$$

15.2　周囲 8 セルを読み取り生存数を数える

```
import Base

# In an 8x8 board, x is the column and y is the row (both 0..7).
# This helper assumes the caller has already checked the range.
def index(x: U32, y: U32) -> U32:
  (y * 8 + x : U32)

# Read one cell from an 8x8 torus.
# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
  Array.get(U32, b, index(x, y))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
```

```
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def blinker() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0, index(3, 3), 1)
  b2 = Array.set(U32, b1, index(4, 3), 1)
  Array.set(U32, b2, index(5, 3), 1)

def count_from_pair(r: Array<U32> & (U32 & U32)) -> U32:
  (board, values) = r
  (n, alive) = values
  n

def main() -> U32:
  count_from_pair(neighbors(4, 4, blinker()))
```

Listing 15.1: 近傍数を足す（06_neighbors.bend）

neighbors は (board,sum) を返します。元の配列を b1,b2,...,b8 へ順に渡すので、8 回の読み取りで board を違法に共有しません。+xl、+xr、+yt、+yb は、計算した座標値を複数の読み取りに使うために必要。

要点

近傍の重複を防ぐには、3×3 の中心 (x, y) を除く 8 箇所を列挙し、同じ (x, y) を含めていないか紙に書いて点検します。**旧盤面から読み、読んだ配列を戻す**という型付きの設計は、この過程の所有権をはっきりさせる。

15.3 境界にいるセルを手計算する

例えば $(0,0)$ の 8 近傍は $(7,7),(0,7),(1,7),(7,0),(1,0),(7,1),(0,1),(1,1)$。四隅のセルで座標の循環をテストします。$(4,4)$ など内部座標だけを調べても境界バグは見つからない。

ハンズオン／確認問題

(1) $(0,0)$ の 8 近傍座標を書き出してみましょう。**(2)** 初期グライダーに対して $(1,1)$ の生存近傍数を手計算してみましょう。**(3)** コード内で `+xl` を外して、どの使用が重複するか確認してみましょう。**(4)** 06_neighbors.bendで `neighbors(4,4,...)` の期待値は 3 になることを考えてみましょう。

Chapter 16

全セルを同期的に更新する

いよいよ 1 世代の更新を実装します。**同期更新（synchronous update）**とは、次の盤面を計算するとき、すべてのセルが同じ古い盤面を読む方法です。

16.1　更新に必ず二枚の盤面を使う

正しい更新では、時刻 t の状態 B_t だけを見て B_{t+1} を作る。例えば上から順に古い配列に直接書き込むと、後で処理するセルが更新済みの近傍値を読みます。その瞬間、同時更新ではなく走査順依存の別のゲームになります。

$$B_{t+1}(x,y) = F\Big(B_t(x,y), \sum_{(u,v)\in N(x,y)} B_t(u,v)\Big).$$

16.2　再帰処理が終わる条件を決める

燃料（fuel）とは、再帰を続ける残り回数を表す値を比喩的に呼んだものです。燃料が 0 になれば止まる、と決めれば無限に繰り返さないことを説明しやすくなります。

`fill(todo, i, old, out)` の `todo` は `Nat` で、残り何セル更新するかを表します。`match todo` で `0` なら出力盤面を返し、`1n+p` なら 1 セル更新して `fill(p,...)` を呼びます。再帰が必ず止まることを `todo` の減少から説明できます。

```
import Base

# In an 8x8 board, x is the column and y is the row (both 0..7).
# This helper assumes the caller has already checked the range.
def index(x: U32, y: U32) -> U32:
  (y * 8 + x : U32)

# Fixed 8x8 toroidal Game of Life, B3/S23.
# Cells are U32 values, maintained in {0,1} by seed and rule.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
```

```
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
  Array.get(U32, b, index(x, y))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0, index(1, 0), 1)
  b2 = Array.set(U32, b1, index(2, 1), 1)
```

```
  b3 = Array.set(U32, b2, index(0, 2), 1)
  b4 = Array.set(U32, b3, index(1, 2), 1)
  Array.set(U32, b4, index(2, 2), 1)

# Decreasing Nat argument first: accepted recursive shape.
# Carry a board, neighbour count, and original cell state together.
# Destructuring is allowed here because r is a parameter.
def fill(todo: Nat, +i: U32, r: Array<U32> & (U32 & U32), out: Array<U32>) -> Array<U32>:
  match todo:
    case 0n:
      out
    case 1n+p:
      (old, pair) = r
      (n, alive) = pair
      next = rule(alive, n)
      out1 = Array.set(U32, out, i, next)
      +j = (i + 1 : U32)
      x = (j % 8 : U32)
      y = ((j / 8 : U32) % 8 : U32)
      fill(p, j, neighbors(x, y, old), out1)

def step(board: Array<U32>) -> Array<U32>:
  fill(64n, 0, neighbors(0, 0, board), [0 : U32*64n])

def evolve(steps: Nat, board: Array<U32>) -> Array<U32>:
  match steps:
    case 0n:
      board
    case 1n+p:
      evolve(p, step(board))

# Reading consumes and returns the array, which can be dropped here.
def value_from_pair(r: Array<U32> & U32) -> U32:
  (b, v) = r
  v

def main() -> U32:
  value_from_pair(Array.get(U32, evolve(4n, glider()), index(2, 3)))
```

Listing 16.1: 完成した純粋な更新核（07_life_core.bend）

16.3 ソースを読む順番

1. `glider()` は五つの座標を `index(x,y)` で指定して初期配置を作ります。
2. `step(board)` は旧盤面と全 0 の新盤面を用意します。
3. `fill(64n,0,old,out)` は 64 セルを順番に処理します。ここは配列全体の走査なので、内部の添字 `i` を使います。
4. `neighbors` と `at` は座標を受け取り、`index(x,y)` を介して旧盤面のセルを読みます。
5. `rule` で次状態を決め、`Array.set` で新盤面の対応位置に保存します。
6. `fill` が残りを処理します。旧盤面は最後まで変更しません。

この走査は計算量 $O(WH)$、追加記憶量 $O(WH)$ です。8 方向参照の定数が大きいため、厳密な性能比較では「セル 1 個当たりの読み込み・格納」を数える。

16.4 別の実装と結果を比較する

同じ `neighbors` を使ってテスト期待値を作ると、その関数に同じ誤りがあった場合に見逃します。したがってscripts/reference.pyは、別の記述・別の実装言語で定義した生存セルの `set` から計算します。別の方法で得た期待値と照合すると、見落としていた誤りを発見しやすくなります。

ハンズオン／確認問題

(1) `fill` の先頭引数を `U32` のカウンタに置き換えると停止性検査にどんな障害があるか。(2) `step` を旧配列 1 枚への更新に変え、グライダーの軌跡がどこから破綻するか観察してみましょう。(3) `step(glider())` の 8×8 値をオラクルと照合してみましょう。

Chapter 17

複数世代の実行とターミナル描画

1 世代だけでなく複数世代を動かし、ターミナルへ表示します。計算と表示を別々の関数にすると、後から見た目だけを変更できます。

17.1　世代を繰り返す関数

evolve(steps,board) は、0 ステップならそのまま返し、正の Nat なら step を呼んでから evolve(p,...) を呼びます。ここで p が 1 減った残りの自然数になっています。型検査は停止の根拠を要求するため、evolve の再帰が無限にループする例は通常許されません。

17.2　出力を作る関数は更新と分ける

一つの行は 8 セルを読み文字列化し、8 行を結合します。render_cells と render_rows は (board,String) を返します。読み出しに使った配列を次の処理へ回せるようにするためです。IO.print は show だけが持つ。

```
import Base

# In an 8x8 board, x is the column and y is the row (both 0..7).
# This helper assumes the caller has already checked the range.
def index(x: U32, y: U32) -> U32:
  (y * 8 + x : U32)

# Same algorithm as 07_life_core, now with a text renderer.
def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

# Each read consumes a board and produces a new board plus a cell value.
def at(b: Array<U32>, x: U32, y: U32) -> Array<U32> & U32:
```

```
  Array.get(U32, b, index(x, y))

# Returned board, neighbour count and original cell value.
def near_9(r: Array<U32> & U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, alive) = r
  (b, (sum, alive))

def near_8(r: Array<U32> & U32, x: U32, y: U32, sum: U32) -> Array<U32> & (U32 & U32):
  (b, v) = r
  near_9(at(b, x, y), (sum + v : U32))

def near_7(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_8(at(b, xr, yb), x, y, (sum + v : U32))

def near_6(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_7(at(b, x, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_5(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_6(at(b, xl, yb), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_4(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_5(at(b, xr, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_3(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_4(at(b, xl, y), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_2(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_3(at(b, xr, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def near_1(r: Array<U32> & U32, +x: U32, +y: U32, +xl: U32, +xr: U32, +yt: U32, +yb: U32, sum: U32) -> Array<U32> &
    (U32 & U32):
  (b, v) = r
  near_2(at(b, x, yt), x, y, xl, xr, yt, yb, (sum + v : U32))

def neighbors(+x: U32, +y: U32, b: Array<U32>) -> Array<U32> & (U32 & U32):
  +xl = ((x + 7 : U32) % 8 : U32)
  +xr = ((x + 1 : U32) % 8 : U32)
  +yt = ((y + 7 : U32) % 8 : U32)
  +yb = ((y + 1 : U32) % 8 : U32)
  near_1(at(b, xl, yt), x, y, xl, xr, yt, yb, 0)

def glider() -> Array<U32>:
  b0 = [0 : U32*64n]
  b1 = Array.set(U32, b0, index(1, 0), 1)
  b2 = Array.set(U32, b1, index(2, 1), 1)
  b3 = Array.set(U32, b2, index(0, 2), 1)
  b4 = Array.set(U32, b3, index(1, 2), 1)
  Array.set(U32, b4, index(2, 2), 1)

# Carry a board, neighbour count, and original cell state together.
# Destructuring is allowed here because r is a parameter.
def fill(todo: Nat, +i: U32, r: Array<U32> & (U32 & U32), out: Array<U32>) -> Array<U32>:
```

```
  match todo:
    case 0n:
      out
    case 1n+p:
      (old, pair) = r
      (n, alive) = pair
      next = rule(alive, n)
      out1 = Array.set(U32, out, i, next)
      +j = (i + 1 : U32)
      x = (j % 8 : U32)
      y = ((j / 8 : U32) % 8 : U32)
      fill(p, j, neighbors(x, y, old), out1)

def step(board: Array<U32>) -> Array<U32>:
  fill(64n, 0, neighbors(0, 0, board), [0 : U32*64n])

# The recursion itself receives the array and the value as a parameter.
def read_cells(todo: Nat, +i: U32, text: String, r: Array<U32> & U32) -> Array<U32> & String:
  match todo:
    case 0n:
      (board, v) = r
      (board, text ++ U32.show(v))
    case 1n+p:
      (board, v) = r
      newer = text ++ U32.show(v)
      read_cells(p, (i + 1 : U32), newer, Array.get(U32, board, (i + 1 : U32)))

def render_cells(n: Nat, +i: U32, board: Array<U32>) -> Array<U32> & String:
  match n:
    case 0n:
      (board, "")
    case 1n+p:
      read_cells(p, i, "", Array.get(U32, board, i))

def read_rows(todo: Nat, +i: U32, text: String, r: Array<U32> & String) -> Array<U32> & String:
  match todo:
    case 0n:
      (board, line) = r
      (board, text ++ line ++ "\n")
    case 1n+p:
      (board, line) = r
      newer = text ++ line ++ "\n"
      read_rows(p, (i + 8 : U32), newer, render_cells(8n, (i + 8 : U32), board))

def render_rows(n: Nat, +i: U32, board: Array<U32>) -> Array<U32> & String:
  match n:
    case 0n:
      (board, "")
    case 1n+p:
      read_rows(p, i, "", render_cells(8n, i, board))

def show_rendered(todo: Nat, +generation: U32, r: Array<U32> & String) -> IO(Unit):
  match todo:
    case 0n:
      (board, picture) = r
      do IO<Unit>:
        IO.print("generation " ++ U32.show(generation))
        IO.print(picture)
        IO.print("done")
    case 1n+p:
      (board, picture) = r
      do IO<Unit>:
        IO.print("generation " ++ U32.show(generation))
        IO.print(picture)
```

```
        show_rendered(p, (generation + 1 : U32), render_rows(8n, 0, step(board)))

def main() -> IO(Unit):
  show_rendered(4n, 0, render_rows(8n, 0, glider()))
```

Listing 17.1: 完成版：5 世代を表示するシミュレータ（08_life_terminal.bend）

17.3 実行する

```
bend src/08_life_terminal.bend --check-only
bend src/08_life_terminal.bend
bend src/08_life_terminal.bend -o build/life
./build/life --threads 1
```

Listing 17.2: 完成版の実行

このプログラムは**世代 0〜4 の 5 枚**を表示します。`IO.print` が末尾の改行を追加するので、盤面の間には空行が入る。`show(5n,0,glider())` を `show(10n,0,glider())` にすれば 10 枚になります。

17.4 期待される世代 4

```
00100000
00010000
01110000
```

Listing 17.3: Python 参照実装が算出した世代 4 の一部

これはグライダーが右下へ 1 マス移動した世代 4 の盤面から 3 行を抜き出したものです。完全な 8 行はscripts/reference.pyで確認できます。

ハンズオン／確認問題

(1) `show` の世代数を 3、5、10 に変更してみましょう。(2) 表示を `1/0` から `#/･` へ変えるための `glyph` 関数を追加してみましょう。(3) 初期配置の章で作った `block()` を使い、静止パターンであることを調べてみましょう。

Chapter 18

テスト設計と失敗の切り分け

要点

CI（継続的インテグレーション）は、GitHub への変更をきっかけにテストを自動実行する仕組みです。テストは計算結果の不一致を検出し、失敗したときには終了コードを 0 以外にして通知します。

動作を確認するための**テスト（test）**を作ります。完成したプログラムと、別方式で作った参照実装の結果を比べます。

18.1　プログラムの正しさをテストで確認する

ライフゲームの出力が期待どおりか確認しましょう。空の盤面や静止する模様など、結果が分かっている入力を用意して比べます。

コンパイラが型の整合性と停止性を確認しても、`neighbors` で中央セルを足してしまう誤りや、`rule` で 2 と 3 を取り違える誤りは型だけでは防げない。少なくとも**局所規則**、**初期配置**、**時刻発展**、**境界条件**の 4 レベルで検査します。

18.2　参照実装を動かす

```
python3 -m unittest discover -s tests -v
python3 scripts/reference.py
python3 scripts/check_bend.py
python3 scripts/check_bend.py --native
```

Listing 18.1: 付属の Python 独立テスト（標準ライブラリのみ）

最後の 2 行は Bend が導入されている場合の任意の**統合テスト**です。`check_bend.py` は Bend の表示から 5 枚の盤面を取り出し、Python オラクルの出力と改行込みで比較します。失敗したら diff を表示します。

18.3 用意された参照テスト 6 件

テスト	検査内容
`test_empty_stays_empty`	空盤面は空のまま。
`test_block_is_still_life`	ブロックは 1 世代後も変化しません。
`test_blinker_oscillates`	ブリンカーは横 → 縦 → 横の 2 周期。
`test_glider_moves...`	4 世代後、グライダーが斜めに 1 進みます。
`test_alive_count...`	グライダーの生存数は 12 世代を通じて 5。
`test_transcript...`	出力の枚数・終端テキストの形。

要点

Python の参照テスト 6 件と、Bend 2.0.37 での 5 世代全セルの一致確認は CI で成功しています。コンパイラを更新した場合は、テストを再実行しましょう。

ハンズオン／確認問題

(1) トーラス端をまたぐブリンカーを設定し、2 周期を検証するテストを書いてみましょう。(2) `rule` の「2 のとき現状維持」を意図的に崩し、どのテストが落ちるかを報告してみましょう。(3) 手計算・Python・Bend の 3 列からなる検証表を作ってみましょう。

Chapter 19

独立した計算を CPU で並列に実行する

これまでの `step` は 1 世代を順番に更新しました。ここでは二つの独立した計算を同時に進める**並列計算（parallel computation）**を学びます。

19.1　二つの関数呼び出しを同時に始める

手続き型言語では、一つの関数の結果を待ってから次の関数を呼ぶ書き方が一般的です。Bend 2 では、互いに独立した呼び出しを次のように横に並べます。

```
a b = f(x) g(y)
# a と b の両方が得られた後で次の処理へ進みます。
```

Listing 19.1: 二つの独立した呼び出し

これを**並列呼び出し（parallel call）**と呼びます。**二つの処理が同じ値を不正に共有しないこと**は Bend の純粋性とアフィン性に支えられます。仕事量が大きく違うと、片方が終わるまで待つ時間が増えるので、負荷の均等さも重要です。

```
import Base

# A genuine balanced fork/join benchmark of independent calls.
def pow2(+depth: Nat) -> U32:
  match depth:
    case 0n:
      1
    case 1n+p:
      left right = pow2(p) pow2(p)
      (left + right : U32)

def main() -> U32:
  pow2(12n)
```

Listing 19.2: 二つに分割しながら値を足す（09_parallel_tree.bend）

`pow2(12n)` の結果は $2^{12} = 4096$ です。`match` で再帰が終了する条件を定め、二つの再帰呼び出しを同じ深さで実行しています。

19.2　ライフゲームを二つ同時に計算する

ライフゲームには、**別々の初期盤面から始まる二つのシミュレーション**という独立した仕事があります。異なる入力を使うので、所有権を共有しなくても並列に実行できます。

```
import Base
import ./life_core.bend as Life

# Deconstruct only function parameters, not locally computed expressions.
def add_cells(a: Array<U32> & U32, b: Array<U32> & U32) -> U32:
  (a_board, a_value) = a
  (b_board, b_value) = b
  (a_value + b_value : U32)

# Independent simulations can run at the same time.
# This is batch parallelism, not cell-level parallelization.
def main() -> U32:
  left right = Life.evolve(4n, Life.glider()) Life.evolve(4n, Life.glider())
  add_cells(Life.at(left, 2, 3), Life.at(right, 2, 3))
```

Listing 19.3: 独立した二つの盤面を並列に更新する（31_parallel_life_pair.bend）

この例は**複数盤面の同時計算**です。一つの盤面の 64 セルすべてを並列化しているわけではありません。`Array<U32>` を再利用可能な値として二つの処理に同時に渡すことはできないので、単一盤面内を並列化する場合には入力の分割や複製を工夫する必要があります。

```
bend src/09_parallel_tree.bend -o build/parallel_tree
./build/parallel_tree --threads 1
./build/parallel_tree --threads 4
```

Listing 19.4: ネイティブ CPU 実行でスレッド数を比較する

スレッド（thread）とは、CPU 上で仕事を実行する単位です。結果が同じことを確認した後で、所要時間を測りましょう。小さな入力では並列化の管理にかかる時間が勝つこともあります。

ハンズオン／確認問題

`pow2(10n)` と `pow2(12n)` を実行して、結果を予想してみましょう。二つの独立した初期盤面について、並列計算した結果と逐次計算した結果が一致するか確かめましょう。

Chapter 20

GPU で計算を実行する

GPU（graphics processing unit）は多数の計算を同時に実行できる装置です。Bend 2 では、並列な関数呼び出しの計算先を指定するための短い記法があります。

20.1　関数名の後ろの! を読む

```
value_cpu = pow2(12n)
value_gpu = pow2!(12n)
```

Listing 20.1: 通常の呼び出しと GPU 呼び出し

! は論理否定ではなく、**GPU 呼び出し（GPU call）**を表します。GPU が利用できない環境では CPU 側で実行されることがあります。JavaScript 実行環境ではこの指定が逐次実行として扱われる場合があります。**GPU 呼び出しを書くだけで必ず速くなるわけではありません**。

```
import Base

# GPU offloading is requested at the root, not with an explicit kernel.
def pow2(+depth: Nat) -> U32:
  match depth:
    case 0n:
      1
    case 1n+p:
      left right = pow2(p) pow2(p)
      (left + right : U32)

def main() -> IO(Unit):
  IO.print(U32.show(pow2!(12n)))
```

Listing 20.2: GPU 実行を要求する例（10_gpu_tree.bend）

```
import Base
import ./life_core.bend as Life

# The helper matches its parameter, not the result of a nested call.
def cell_from_result(r: Array<U32> & U32) -> U32:
  (board, value) = r
  value

# GPU dispatch syntax; a sequential Life.step is NOT a
# cell-parallel GPU Life implementation or a speedup guarantee.
def main() -> IO(Unit):
```

```
  board = Life.evolve!(4n, Life.glider())
  IO.print(U32.show(cell_from_result(Life.at(board, 2, 3))))
```

Listing 20.3: ライフゲームの計算を GPU へ渡す書き方（32_gpu_life_call.bend）

後者の `evolve!` は **GPU への実行指定を学ぶための例**です。その内部の `step` は逐次処理なので、セル単位の GPU 並列版が完成したことにはなりません。単一盤面を真に並列更新するには、旧盤面の各セルを安全に読むデータ分割と、計算後の盤面を統合する設計が必要です。

20.2 GPU の有無を分けて確かめる

```
bend src/10_gpu_tree.bend -o build/gpu_tree
./build/gpu_tree
./build/gpu_tree --gpu off
```

Listing 20.4: GPU 実行と CPU 代替実行

ネイティブ実行に必要な GPU ドライバや対応コンパイラは環境によって異なります。まず出力が 4096 で一致するか確かめ、その後で速度を比べましょう。8×8 の小さな盤面では、GPU の起動コストが大きくなる可能性があります。

ハンズオン／確認問題

CPU 実行と GPU 指定の違いを説明してみましょう。! を付けたプログラムについて、計算結果が同じかどうか調べてみましょう。速度を測った場合は、使用したハードウェアと測定方法も記録しましょう。

Chapter 21

ライフゲームの規則を形式的に証明する

テストは選んだ入力を調べます。一方、**形式的証明（formal proof）**は「すべての対象入力について成り立つ」という主張を、検査できる形で示します。Bend 2 では `law` と `def` でそれを記述できます。

21.1　法則と証明を対応させる

例えば「近傍が 3 つなら必ず次の状態は生存」という**命題（proposition）**を考えます。

```
import Base
law three_is_alive:
  for cell: U32
  {rule(cell, 3) == 1 : U32}

def three_is_alive(cell):
  {==}
```

Listing 21.1: 法則と証明を記述する最小例

上の例は `rule` の定義を省略しています。実際に実行する際は次のファイルを使用してください。

```
import Base

def choose(is_three: Bool, is_two: Bool, alive: U32) -> U32:
  match is_three:
    case True{}:
      1
    case False{}:
      match is_two:
        case True{}:
          alive
        case False{}:
          0

def rule(alive: U32, +n: U32) -> U32:
  choose(U32.is_eq(n, 3), U32.is_eq(n, 2), alive)

law three_is_alive:
  for cell: U32
  {rule(cell, 3) == 1 : U32}

def three_is_alive(cell):
  {==}

law two_preserves:
```

```
  for cell: U32
  {rule(cell, 2) == cell : U32}

def two_preserves(cell):
  {==}

law zero_is_dead:
  for cell: U32
  {rule(cell, 0) == 0 : U32}

def zero_is_dead(cell):
  {==}

# A proof file can omit main. Run: bend 11_rule_proofs.bend --verdict
```

Listing 21.2: 生死規則に対する三つの形式的証明（11_rule_proofs.bend）

`for cell: U32` は、対象を**すべての `U32` の値**に広げます。`{a == b : U32}` は二つが等しいという**等式の型（equality type）**で、`{==}` は左右が計算により同じ式になることを示す証明です。

このファイルにある三つの法則の意味は、近傍数が 3 なら出力が 1、2 なら元の状態を保つ、0 なら出力が 0、です。これらは固定した 18 例より広い条件について証明しますが、**B3/S23 のすべての分岐が証明済みという意味ではありません**。

21.2　帰納法と証明の検査

数学的帰納法（mathematical induction）は、最初の値で成り立つことと、一つ小さい値で成り立つなら次の値でも成り立つことを示す方法です。Bend では `match` と再帰を使って書けます。

```
import Base

law plus_zero:
  for x: Nat
  {Nat.add(x, 0n) == x : Nat}

def plus_zero(x):
  match x:
    case 0n:
      {==}
    case 1n+p:
      %plus_zero(p) : {1n+Nat.add(p, 0n) == 1n+_ : Nat}
      {==}

def main() -> {Nat.add(2n, 0n) == 2n : Nat}:
  plus_zero(2n)
```

Listing 21.3: 自然数を用いた帰納法と書き換え（24_proof_induction.bend）

`%plus_zero(p)` は、再帰で得た証明を用いた**等式の書き換え（rewriting）**です。`?TODO` のような未完成の箇所を残したものを証明完了とは扱いません。

```
bend src/11_rule_proofs.bend --verdict
bend src/24_proof_induction.bend --verdict
```

Listing 21.4: Bend の証明検査

21.3　盤面全体の仕様を証明するには

一つのセルの `rule` を証明しても、`neighbors` がちょうど 8 方向を数え、`step` が旧盤面だけを使って 64 セルを正しく更新したことまでは分かりません。全体を証明するには、**添字変換、近傍の列挙、全セルの同期更新**について別の法則を作り、組み合わせる必要があります。本書では、局所規則の形式的証明と、盤面全体の実行テストを区別します。

ハンズオン／確認問題

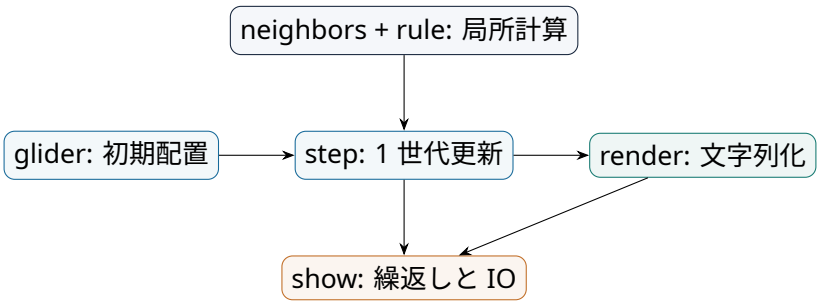
近傍数が 1 のとき必ず死ぬことを表す `law one_dies` を追加し、`{==}` で証明してみましょう。次に、近傍を数える関数の正しさを法則として書くには、どの情報が必要か考えてみましょう。

Chapter 22

完成版の設計をレビューする

いったん完成したプログラムを読み直します。**保守性（maintainability）**とは、後から機能追加や修正をしやすいことです。

22.1　関数が担う責務



`rule` は 1 セルの遷移、`neighbors` は 8 セルの集計、`step` は盤面全体の遷移、`render` は表示、`show` は実行制御を担当します。それぞれの関数の役割が明確だと、テストで誤りの場所を絞り込めます。

22.2　保守性チェック

1. **定数が散らばっている**: 8 と 64 が複数箇所にあります。定数に名前を付けると変更箇所を減らせます。
2. **0/1 の意味が型に表れていない**: 新しく生死状態を表す型を定義する方法もあります。
3. **表示が計算から独立**: 文字列化を別の関数にしているので、セルの表現を変更しても世代更新の関数を修正せずに済みます。
4. **出力枚数は固定**: 表示する世代数は `show` への引数で変更できます。

ハンズオン／確認問題

`glider`、`step`、`render`、`show` について、引数・返り値・画面表示を行うかどうかを表にまとめてみましょう。`rule` の実装を変更したとき、どのテストを実行すればよいでしょうか。

PART III

演習の解答と実行リファレンス

この Part の目次

23 重要演習の解答と検証方法 53

24 環境・コマンド・参照先 55

Chapter 23

重要演習の解答と検証方法

ここからは解答・付録です。自力で解いてから照合し、単に答えが一致したかだけでなく、理由も説明できるか確認してください。

23.1　18 ケースの局所規則

n	0	1	2	3	4	5	6	7	8
$F(0,n)$	0	0	0	1	0	0	0	0	0
$F(1,n)$	0	0	1	1	0	0	0	0	0

23.2　代表的な添字

$(0,0)\mapsto 0$, $(7,0)\mapsto 7$, $(0,7)\mapsto 56$, $(7,7)\mapsto 63$。$(2,3)\mapsto 26$。$(7,0)$ の右は $(0,0)$ で、$(0,1)$ ではありません。

23.3　グライダーの変位

世代 0 の集合は $\{(1,0),(2,1),(0,2),(1,2),(2,2)\}$。4 回の更新後は $\{(2,1),(3,2),(1,3),(2,3),(3,3)\}$。`tests/test_reference.py` にはこの性質を全セル集合として比較するテストがあります。

23.4　近傍読み出し回数

1 世代で $N = WH$ 個のセルを更新するなら、現行の各セルは `neighbors` の 8 回と現在セルの 1 回で、合計 $9N$ 回の `Array.get` を使います。`Array.set` は N 回。計算量の章の「8 方向参照」と現在セルの読取りを区別すること。

23.5 典型的なバグ

症状	疑う箇所
盤面右端付近だけ違う	x 方向のラップと一次元添字の混同。
ブリンカーが 1 周期で崩れる	旧世代を更新しながら後続の近傍を読んでいる。
すべてのセルが 1 になる	自分自身を近傍として含めた、あるいは条件式の 3 と 2 を間違えた。
`used more than once`	配列を戻り値として引き継いでいない、または `+` の不足。

Chapter 24

環境・コマンド・参照先

困ったときに参照するコマンドをまとめています。**再現性（reproducibility）**は、同じ条件で結果を再確認できる性質です。

24.1　コマンド早見表

コマンド	目的
`bend version`	バージョン表示。
`bend guide`	インストール版に一致したガイド。
`bend base`	標準ライブラリの参照。
`bend base Array`	配列周辺の API を個別に調査。
`bend file.bend --check-only`	型・停止性などのチェックだけ実施。
`bend file.bend`	チェックと実行／正規化。
`bend file.bend -o program`	ネイティブバイナリの生成。
`python3 -m unittest discover -s tests`	Python 参照系の自己テスト。

24.2　主要資料

- https://github.com/bendlang/bend
  公式リポジトリ、README、guide、Base、リリース。
- https://github.com/bendlang/bend/releases（公式の最新版・リリース日時）。
- https://bend2.dev/（非公式だが例題の豊富な学習リファレンス。公式 `bend guide` が優先）。
- https://nohzafk.github.io/bend2-from-zero/（Bend2 の失敗例とライフゲームを含む独立教材。実装上の参考情報）。

24.3　検証ログの書式

```
Date: YYYY-MM-DD
OS / Arch:
Bend version:
Bend native clang:
```

```
  Commands:
    bend src/08_life_terminal.bend --check-only
    python3 scripts/check_bend.py
  Result: PASS / FAIL / SKIP
  Error output (if any):
  Changes made after checking:
```

Listing 24.1: VERSION_LOG.md の記入例（数値は実測値に置換）

24.4 最後の総合チェック

1. **データ型**: U32、Nat、String、ユーザー定義型の違いを説明できます。
2. **関数**: 引数・返り値と match による場合分けを書けます。
3. **データ構造**: 配列の要素を読み書きし、所有権を戻り値で受け渡せます。
4. **関数型の考え方**: 高階関数、クロージャ、Maybe による失敗の扱いを説明できます。
5. **ライフゲーム**: 生死規則と同期更新を実装し、動作をテストで確かめられます。